

# Memory-Centric Agentic Inference — cycles 1-3

2026-05-11

## Contents

|                                                      |          |
|------------------------------------------------------|----------|
| <b>Memory-Centric Agentic Inference — cycles 1-3</b> | <b>1</b> |
| Abstract . . . . .                                   | 1        |
| Introduction . . . . .                               | 2        |
| Methodology . . . . .                                | 2        |
| Results . . . . .                                    | 2        |
| Discussion . . . . .                                 | 9        |
| Conclusions and Recommendations . . . . .            | 9        |
| References . . . . .                                 | 10       |
| Appendix: Implementation Details . . . . .           | 10       |

## Memory-Centric Agentic Inference — cycles 1-3

### Abstract

Cycles 1-3 established the foundation for a memory-centric architecture investigation for agentic large language model (LLM) inference. The work did not attempt a generic literature survey or a hardware-calibrated performance model. Instead, it built a sequence of inspectable artifacts: a workload and memory-object taxonomy, a symbolic lifetime and reuse-distance model, and a heterogeneous memory cost model with executable generation scripts and validation outputs.

The validated progression is:

1. M-TAX-1: define what memory objects exist and which workloads exercise them.
2. M-LIFE-1: define how those objects are born, reused, retained, merged, discarded, or made durable.
3. M-COST-1: define how lifetime and reuse translate into residency, movement, bandwidth-delay, recomputation, correctness-risk, energy-proxy, and dollar-proxy terms across memory tiers.

The central finding from these cycles is that agentic inference memory pressure is not reducible to model weights plus linear key-value (KV) cache growth. KV cache remains important, but branch state, verifier state, tool outputs, trajectory logs, retrieved context, semantic caches, and durable workspace artifacts introduce different lifetime and reuse mechanisms. These mechanisms depend on branch survival, verifier delay, merge behavior, provenance needs, auditability, and workspace retention horizons. This finding was validated by the cycle auditors as a defensible basis for the next milestone, M-SIM-1, which should compare memory placement policies using the generated taxonomy, lifetime model, and cost scenario tables.

No external hardware constants or production trace measurements were used in these cycles. All tier values, energy proxies, and dollar proxies are explicitly symbolic or synthetic placeholders.

## Introduction

The project investigates whether future infrastructure for agentic LLM inference should be designed around memory movement, placement, reuse, compression, and lifetime management rather than arithmetic throughput alone. The directive framed the problem around long-running agentic systems: systems that branch, verify, call tools, resume prior work, maintain durable workspaces, and merge results across agents.

The initial plan of record decomposed the work into six milestones: taxonomy, lifetime modeling, cost modeling, simulation, scheduling abstraction evaluation, and architecture synthesis. Cycles 1-3 completed the first three milestones. The researcher sessions deliberately ordered the work so that later executable models would not confuse workload shape with memory hierarchy policy:

- Cycle 1 built the memory-object and workload vocabulary.
- Cycle 2 turned that vocabulary into symbolic lifetime and reuse equations.
- Cycle 3 attached memory-tier and cost terms to the lifetime variables.

This report consolidates only cycles 1-3. Later milestones, including workload simulation, scheduling comparison, and architecture proposal, remain open.

## Methodology

The work used an engineering-style staged model. Each cycle had a researcher brief, a worker implementation, and an auditor validation.

The researcher briefs set falsifiable objectives and defined the artifact contract for each milestone. The workers then created markdown specifications, CSV schemas, Wolfram Language scripts, Python plotting scripts, and generated plots. The auditors checked required artifacts, regenerated scripts, inspected generated data, and either validated the milestone or patched a defect before validation.

The reporting sources for this document were:

- Cycle 1 researcher session 5383a712-2995-434a-a38d-9597c7edc476
- Cycle 1 worker session ab4104f2-ba5e-40cf-84e8-f5f31a4ebc37
- Cycle 1 auditor session 132d1bc4-999f-4f8f-9f33-414f602a05de
- Cycle 2 researcher session 449332db-21d2-4f4c-a52b-d3979d670086
- Cycle 2 worker session d0a5058a-b9e9-4548-93e7-f8e17099361b
- Cycle 2 auditor session 389bf178-874c-43ed-bde4-5860bfeecced
- Cycle 3 researcher session 5a5977be-403c-41f6-885c-1f5583b9695e
- Cycle 3 worker session 37e7fd2b-5a03-4a41-be70-c373d8d4604b
- Cycle 3 auditor session 25575eac-5684-480f-973b-5a9ad204f9fb

A gap in the record is that no dedicated `search_sessions` tool was available in this reporting environment. The direct session IDs supplied in the prompt were fetched from the local SQLite session database instead. No `REFERENCES.md` file exists in the workspace, and the artifacts state that no external sources were used in these cycles.

## Results

### Cycle 1: Workload and Memory-Object Taxonomy

Cycle 1 completed `M-TAX-1`, the baseline taxonomy. The researcher brief required at least six workload classes and seven memory-object classes, with falsification criteria and downstream fields for later modeling. The worker produced nine workload classes and eleven memory-

object classes.

The workload classes are:

- single-turn chat
- multi-turn chat
- batch summarization
- retrieval-augmented generation (RAG)
- code-agent loop
- tool-using research agent
- verification-heavy agent
- multi-agent branch/merge run
- offline inference

The memory-object classes are:

- weights
- KV cache
- prefix cache
- retrieved context
- tool output
- intermediate scratch
- branch state
- verifier state
- trajectory log
- durable workspace
- semantic cache entry

The key decision in this cycle was to treat workload names as insufficient. The taxonomy instead describes each workload by memory objects touched, dominant lifetime, reuse mode, branch/merge behavior, and whether arithmetic-throughput-only modeling is likely to be sufficient. This decision came from the cycle 1 researcher brief and was implemented in `memory-centric-agentic/taxonomy.md`, `memory-centric-agentic/memory_objects.csv`, and `memory-centric-agentic/workload_classes.csv`.

The taxonomy preserves negative controls. Single-turn chat, batch summarization, and offline inference were marked as workloads where memory-centric placement should have limited additional benefit unless shared prefixes, weight residency, or transient KV capacity dominate. This matters because it prevents the project from hard-coding every workload as support for the memory-centric thesis.

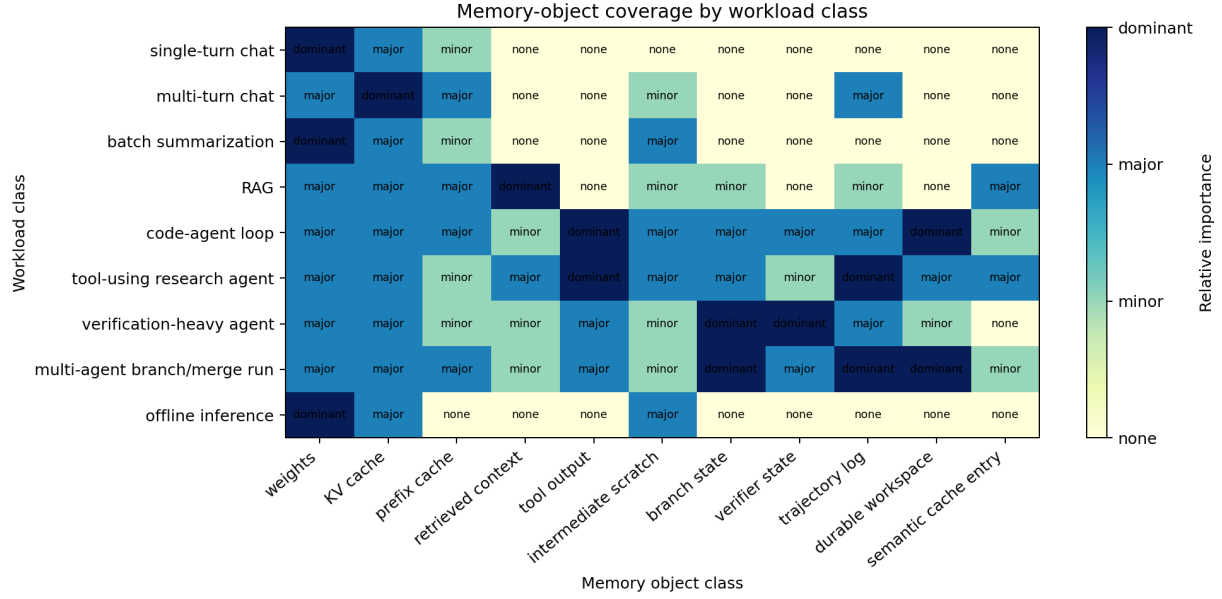
The auditor validated M-TAX-1 in session 132d1bc4-999f-4f8f-9f33-414f602a05de. The audit found no critical or moderate defects. It confirmed that required artifacts existed, all 11 memory objects had required fields, all 9 workloads had hypotheses and falsification signals, every memory object appeared in the coverage matrix, control cases were present, and the heatmap regenerated as a valid nonblank PNG.

## Cycle 2: State Lifetime and Reuse-Distance Model

Cycle 2 completed M-LIFE-1, converting the taxonomy into symbolic lifetime and reuse equations. The worker represented each memory object as:

$$O_i = \{class_i, S_i(t), b_i, e_i, R_i(t), P_i(t), C_{evict_i}\}$$

Here,  $S_i(t)$  is object size over time,  $b_i$  and  $e_i$  are birth and death events,  $R_i(t)$  is the reuse process,  $P_i(t)$  is symbolic placement, and  $C_{evict_i}$  is the eviction consequence class.



**Figure 1:** Heatmap showing which memory-object classes dominate which workload classes; values are categorical importance levels: none, minor, major, dominant.

The model defined expected live bytes as:

$$E[L_i(t)] = E[S_i(t) * 1(b_i \leq t < e_i)]$$

It also defined symbolic retained value as:

$$V_i(t) = \text{Pr}(\text{reuse}_i \text{ before eviction}) * C_{\text{recompute}_i} + \text{Pr}(\text{correctness\_sensitive}_i) * C_{\text{loss}_i} - C_{\text{residency}_i}$$

The main equations covered:

- capped KV growth:  $L_{KV}(T) = s_{kv} * \text{Min}[T, T_{\text{max}}]$
- finite and pinned prefix reuse
- branch-state live bytes under fanout, survival probability, verifier delay, and merge probability
- durable workspace growth under artifact growth rate and retention horizon

The branch-state equation was:

$$E[L_{\text{branch}}] = f * p_s * s_b * (d_v + p_m * d_{\text{merge}})$$

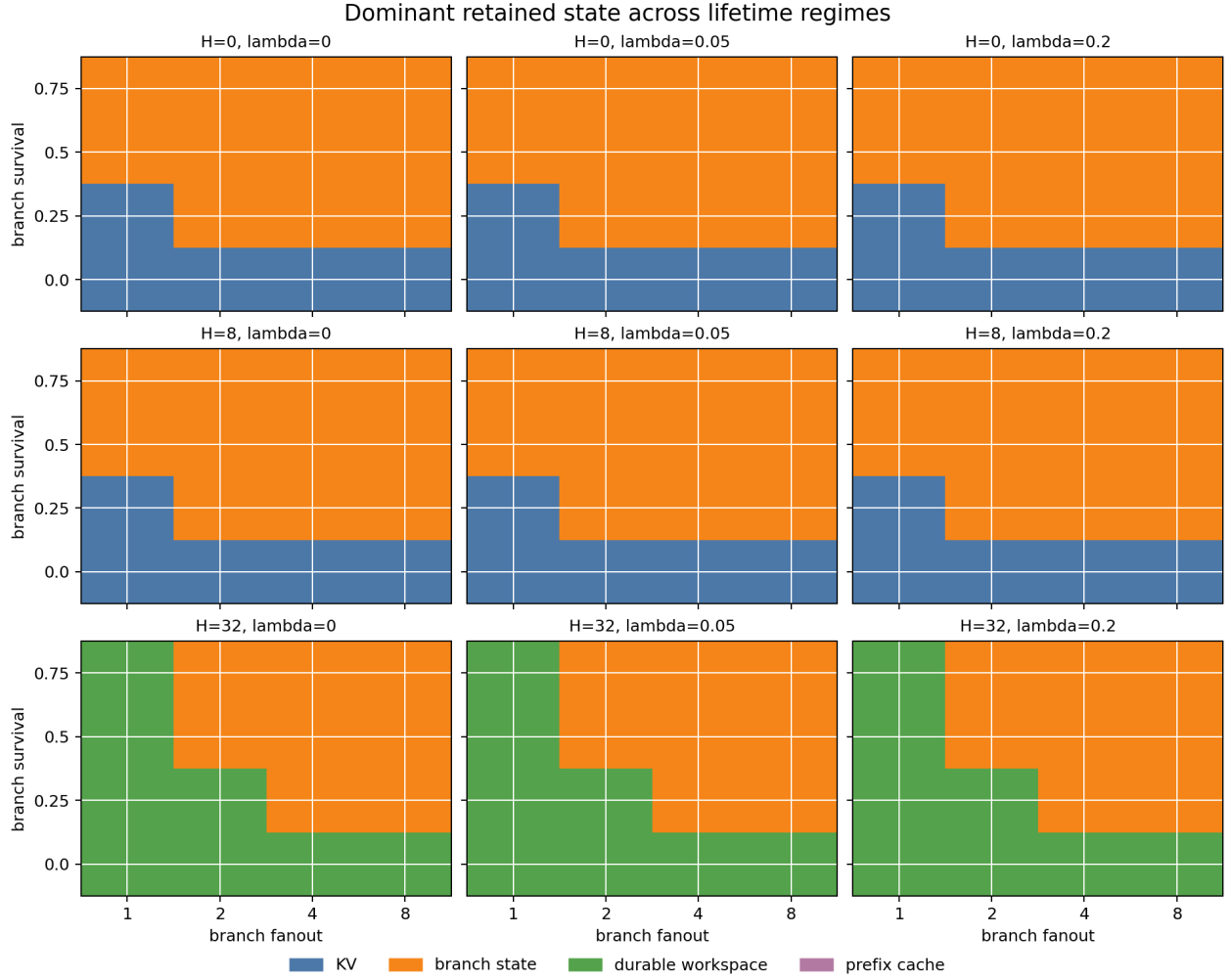
This equation captures the central cycle 2 mechanism: branch memory is not merely token memory. It is retained until verification and may persist into merge/audit state depending on branch survival and merge probability.

The durable workspace equation was:

$$L_{\text{durable}}(H) = D_0 + g_d * H$$

This term is independent of the model context cap unless a later policy summarizes or deletes durable artifacts.

The generated special cases included:



**Figure 2:** Heatmap showing where expected retained state is dominated by KV, branch state, or durable workspace state across branch fanout, branch survival probability, prefix reuse rate, and durability horizon.

| Expression               | Special point            | Symbolic result                                      | Interpretation                                                        |
|--------------------------|--------------------------|------------------------------------------------------|-----------------------------------------------------------------------|
| linear KV growth         | $T = T_{\max}$           | $skv \cdot T_{\max}$                                 | KV live bytes saturate at the context cap.                            |
| prefix reuse             | $\lambda = 0$            | $-C_{\text{residency}}$                              | No reuse leaves only residency cost.                                  |
| prefix reuse probability | infinite or pinned reuse | 1                                                    | A pinned reusable prefix has hit probability one.                     |
| branch live bytes        | $p_s = 0$                | 0                                                    | No surviving branches means no retained branch state.                 |
| branch live bytes        | $p_m = 0$                | $dv \cdot f \cdot ps \cdot sb$                       | Verifier-delay residency remains even without merge tail.             |
| branch live bytes        | $f = 1$                  | $(dv + d_{\text{merge}} \cdot pm) \cdot ps \cdot sb$ | Single-candidate verification remains, but fanout scaling disappears. |
| durable workspace        | $g_d = 0$                | 0                                                    | Zero artifact growth leaves initial workspace bytes only.             |

The auditor validated M-LIFE-1 in session 389bf178-874c-43ed-bde4-5860bfeecced. The audit found no critical or moderate defects. It confirmed that the Wolfram script ran from the workspace root, generated 9 special-case rows and a 144-row regime grid, and produced a nonblank plot. It also confirmed that the model includes an ordinary-control row, 90 branch-dominated rows, and 24 durable-workspace-dominated rows.

### Cycle 3: Heterogeneous Memory Cost Model

Cycle 3 completed M-COST-1, adding a tier and consequence cost layer to the taxonomy and lifetime model. The researcher brief required the worker to avoid numerical overclaiming. As a result, all tier constants are symbolic or synthetic placeholders.

The core cost expression is:

$$C_{\text{total}}(i, k, p) = C_{\text{residency}} + C_{\text{transfer}} + C_{\text{bandwidth\_delay}} + C_{\text{recompute}} + C_{\text{eviction\_loss}} + C_{\text{staleness\_or\_correctness\_risk}}$$

For memory object  $i$ , tier  $k$ , and placement policy  $p$ , the useful retention inequality is:

$$P_{\text{reuse}_i} C_{\text{recompute}_i} + P_{\text{correct}_i} C_{\text{loss}_i} > C_{\text{residency}}(i, k) + C_{\text{transfer}}(i, k) + C_{\text{bandwidth\_delay}}(i, k)$$

This states the placement rule in symbolic form: keeping an object in a useful tier is justified when avoided recomputation plus avoided correctness or audit loss exceeds residency, transfer, and bandwidth-delay cost.

The tier abstraction contains seven tiers:

- HBM/GPU memory
- CPU DRAM
- CXL or pooled memory
- NVMe local storage
- remote object store
- durable workspace store
- semantic cache

The cost model also emits synthetic proxy fields for energy and dollar sensitivity:

$$C_{\text{energy\_proxy}}(i,k) = X_i E_k + E[L_i(t)] H_i E_{\text{idle}_k}$$

$$C_{\text{dollar\_proxy}}(i,k) = E[L_i(t)] H_i D_k$$

These values are dimensionless placeholders. They are not measured joules, watts, dollars, or service costs.

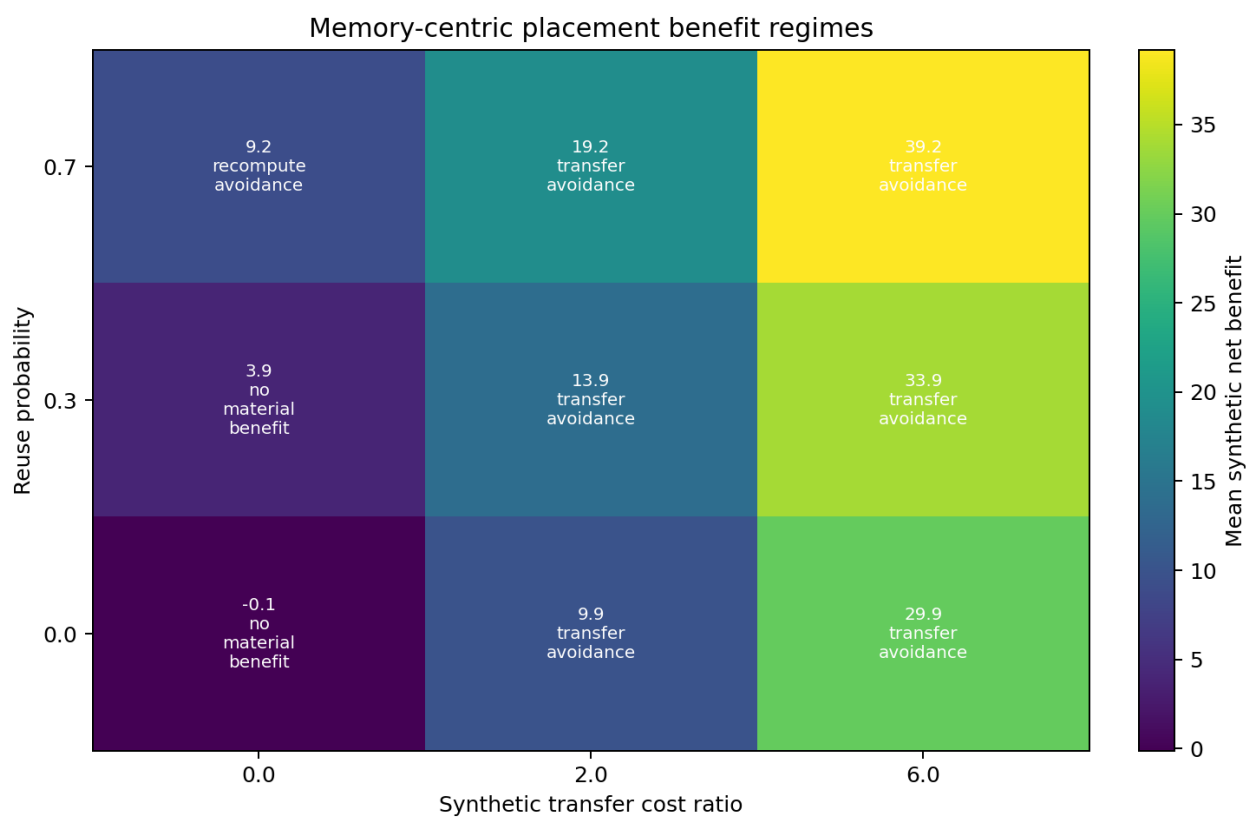
The generated scenario table includes six regimes:

| Scenario                                                | Dominant object   | Dominant term      | Best symbolic tier | Thesis status |
|---------------------------------------------------------|-------------------|--------------------|--------------------|---------------|
| single-turn chat control                                | KV cache          | transfer avoidance | CPU DRAM           | weakened      |
| batch summarization/offline inference control           | weights           | transfer avoidance | CPU DRAM           | weakened      |
| RAG with retrieved-context reuse                        | retrieved context | residency pressure | CPU DRAM           | ambiguous     |
| code-agent loop with tool outputs and durable workspace | tool output       | residency pressure | NVMe local         | ambiguous     |
| verification-heavy agent                                | verifier state    | residency pressure | NVMe local         | ambiguous     |
| multi-agent branch/merge run                            | branch state      | transfer avoidance | NVMe local         | strengthened  |

The most important result is that controls remained weakened while the multi-agent branch/merge run produced a strengthened non-KV case. That non-KV case was driven by branch state and transfer avoidance rather than simple context length.

The cost model boundary cases included zero transfer cost, infinite bandwidth, zero reuse, perfect reuse or pinned prefix, zero durable state, equal tier costs, context cap unsaturated and saturated cases, zero correctness-sensitive loss, and zero recomputation cost.

The auditor validated M-COST-1 in session 25575eac-5684-480f-973b-5a9ad204f9fb after fixing one moderate issue. The issue was that `memory_tiers.csv` exposed energy and dollar symbols, but the executable scenario outputs initially lacked energy and cost proxy fields. The auditor patched `memory-centric-agentic/cost_model.md`, `memory-centric-agentic/cost_assumptions.md`,



**Figure 3:** Heatmap showing synthetic net memory-centric placement benefit by reuse probability and transfer cost ratio; cell labels show dominant benefit driver.



and `scripts/cost_model.wls`, then regenerated outputs. The final scenario table includes populated `energy_proxy_score` and `dollar_proxy_score` columns.

The final validation run passed:

- `timeout 120 wolfram-batch -script scripts/cost_model.wls`
- `python3 scripts/plot_cost_sensitivity.py`
- generated 10 special cases
- generated 6 scenarios
- generated 648 sensitivity rows
- produced a nonblank `data/cost_model_sensitivity.png`

The audit logged one minor issue: `scripts/cost_model.wls` prints `read lifetime regime rows=143` while the prior lifetime grid has 144 data rows. The auditor classified this as a reporting or import-count quirk only, with generated cost artifacts unaffected.

## Discussion

The first three cycles support a specific, bounded version of the memory-centric architecture thesis. The supported claim is not that every inference workload needs agentic memory infrastructure. The controls show the opposite: single-turn chat, batch summarization, and offline inference can remain largely explainable by model weights, prompt/output length, batching, and transient KV behavior.

The supported claim is that long-running agentic workloads introduce memory objects with lifetimes and failure modes that ordinary request-level serving abstractions do not capture well. The clearest examples are:

- branch state retained across speculative paths until verification or merge
- verifier state that affects correctness, not only latency
- tool outputs that serve as evidence and may be expensive or impossible to reproduce
- trajectory logs needed for replay, audit, and resume
- durable workspace artifacts that outlive the model context window
- semantic cache entries whose invalidation risk differs from exact prefix caching

The project's emerging mechanism is:

memory-centric value = avoided recomputation + avoided correctness/audit loss - residency/movement/bandwidth-delay cost

Agentic workloads differ because reuse probability, correctness sensitivity, and lifetime horizon are driven by branch survival, verifier delay, tool provenance, and durable workspace dependencies. These are not always functions of current token count.

The important limitation is that cycles 1-3 remain symbolic and synthetic. They establish structure, executable artifacts, and falsification cases, but they do not yet measure production workloads or calibrate hardware constants. The work is therefore ready for simulation and sensitivity analysis, not for quantitative claims about real datacenter savings.

## Conclusions and Recommendations

Cycles 1-3 completed the foundation needed for policy simulation.

The validated artifacts show that a memory-centric architecture investigation should model at least three layers:

1. Object layer: what state exists, how it is reused, and what fails when it is evicted.

2. Lifetime layer: when state is born, retained, merged, discarded, summarized, or made durable.
3. Tier-cost layer: how state placement affects residency, transfer, bandwidth delay, re-computation, correctness risk, and synthetic energy/cost proxies.

The next milestone should be M-SIM-1: a workload generator and policy comparison harness. The cycle 3 auditor recommended comparing at least three policies:

- HBM-first or ordinary serving baseline
- reuse-aware tier placement
- branch/verifier/durable-aware placement

The simulator should preserve at least one negative/control workload where memory-centric placement does not help. It should also consume `data/cost_model_scenarios.csv`, including the auditor-added `energy_proxy_score` and `dollar_proxy_score` fields.

## References

No external references are cited in this report. The workspace does not contain `REFERENCES.md`, and the cycle artifacts explicitly state that no external hardware, datacenter, production-trace, or energy constants were used in cycles 1-3.

## Appendix: Implementation Details

### Code Organization

The core project artifacts are organized under `memory-centric-agentic/`, with generated data and cross-cutting scripts at the workspace root.

Main narrative/model files:

| File                                                    | Purpose                                                                     |
|---------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>memory-centric-agentic/taxonomy.md</code>         | Narrative workload taxonomy and memory-object role model.                   |
| <code>memory-centric-agentic/assumptions.md</code>      | Cycle 1 assumption labels and guardrails.                                   |
| <code>memory-centric-agentic/lifetime_model.md</code>   | Symbolic object lifetime and reuse-distance model.                          |
| <code>memory-centric-agentic/cost_model.md</code>       | Symbolic/synthetic heterogeneous memory cost model.                         |
| <code>memory-centric-agentic/cost_assumptions.md</code> | Cost model assumption labels, symbolic variables, and deferred assumptions. |

Machine-readable schema and data files:

| File                                                        | Purpose                                                                                                      |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>memory-centric-agentic/memory_objects.csv</code>      | 11 memory-object classes and their lifetime, reuse, placement, compression, eviction, and downstream fields. |
| <code>memory-centric-agentic/workload_classes.csv</code>    | 9 workload classes with controls, hypotheses, and falsification signals.                                     |
| <code>memory-centric-agentic/lifetime_parameters.csv</code> | Lifetime parameter table for all 11 memory-object classes.                                                   |

| File                                               | Purpose                                  |
|----------------------------------------------------|------------------------------------------|
| memory-centric-agentive/memory_tiers.csv           | Seven symbolic/synthetic memory tiers.   |
| memory-centric-agentive/data/taxonomy_coverage.csv | Workload-by-object coverage matrix.      |
| data/lifetime_model_special_cases.csv              | Lifetime model boundary cases.           |
| data/lifetime_regime_grid.csv                      | Lifetime regime grid.                    |
| data/cost_model_special_cases.csv                  | Cost model boundary cases.               |
| data/cost_model_scenarios.csv                      | Six representative cost scenarios.       |
| data/cost_model_sensitivity.csv                    | 648-row synthetic cost sensitivity grid. |

Executable scripts:

| File                                                   | Lines | Purpose                                                                               |
|--------------------------------------------------------|-------|---------------------------------------------------------------------------------------|
| memory-centric-agentive/data/plot_taxonomy_coverage.py | 56    | Regenerates the taxonomy coverage heatmap.                                            |
| scripts/lifetime_model.wls                             | 78    | Generates lifetime special cases and regime grid using Wolfram Language.              |
| scripts/plot_lifetime_regimes.py                       | 91    | Regenerates the lifetime regime dominance heatmap.                                    |
| scripts/cost_model.wls                                 | 146   | Generates cost special cases, scenarios, and sensitivity data using Wolfram Language. |
| scripts/plot_cost_sensitivity.py                       | 75    | Regenerates the cost sensitivity heatmap.                                             |

Figures:

| Figure                                             | Dimensions  | Source                    |
|----------------------------------------------------|-------------|---------------------------|
| memory-centric-agentive/data/taxonomy_coverage.png | 1980 x 990  | plot_taxonomy_coverage.py |
| data/lifetime_regime_plot.png                      | 1890 x 1530 | plot_lifetime_regimes.py  |
| data/cost_model_sensitivity.png                    | 1440 x 936  | plot_cost_sensitivity.py  |

## Test and Validation Results

Cycle 1 validation:

- 11 memory-object classes
- 9 workload classes
- all workload hypotheses and falsification signals present
- all memory objects used in coverage matrix

- control workloads present: single-turn chat, batch summarization, offline inference
- taxonomy heatmap regenerated and checked nonblank
- decision: VALIDATED

Cycle 2 validation:

- Wolfram script generated 9 lifetime special-case rows
- lifetime regime grid generated 144 rows
- grid included 1 ordinary-control row, 90 branch-dominated rows, and 24 durable-workspace-dominated rows
- lifetime plot regenerated and checked nonblank
- decision: VALIDATED

Cycle 3 validation:

- Wolfram script generated 10 cost special-case rows, 6 scenarios, and 648 sensitivity rows
- Python plot script regenerated the cost sensitivity plot
- controls were marked weakened
- multi-agent branch/merge was marked strengthened
- auditor patched missing energy and dollar proxy outputs
- decision: VALIDATED

## File Counts

The current manifest records:

| Category                        | Count |
|---------------------------------|-------|
| Scripts                         | 4     |
| Script lines                    | 390   |
| Markdown model/assumption files | 5     |
| CSV data/model files            | 11    |
| Figures                         | 3     |
| Completed sub-topics            | 3     |

Completed sub-topics:

- M-TAX-1: workload and memory-object taxonomy
- M-LIFE-1: symbolic lifetime and reuse-distance model
- M-COST-1: heterogeneous memory cost model

## Session References

| Cycle | Role       | Session ID                           | Contribution                                                                              |
|-------|------------|--------------------------------------|-------------------------------------------------------------------------------------------|
| 1     | researcher | 5383a712-2995-434a-a38d-9597c7edc476 | Defined M-TAX-1 scope, required artifacts, validation checks, and falsification criteria. |

| Cycle | Role       | Session ID                           | Contribution                                                                                |
|-------|------------|--------------------------------------|---------------------------------------------------------------------------------------------|
| 1     | worker     | ab4104f2-ba5e-40cf-84e8-f5f31a4ebc37 | Built taxonomy markdown, CSVs, assumptions file, coverage matrix, and heatmap.              |
| 1     | auditor    | 132d1bc4-999f-4f8f-9f33-414f602a05de | Validated taxonomy artifacts and recommended moving to lifetime modeling.                   |
| 2     | researcher | 449332db-21d2-4f4c-a52b-d3979d670086 | Defined M-LIFE-1 equations, required special cases, and generated-output contract.          |
| 2     | worker     | d0a5058a-b9e9-4548-93e7-f8e17099361b | Built lifetime model, Wolfram generator, CSV outputs, and dominance heatmap.                |
| 2     | auditor    | 389bf178-874c-43ed-bde4-5860bfecccd  | Validated lifetime equations, special cases, generated grid, and plot.                      |
| 3     | researcher | 5a5977be-403c-41f6-885c-1f5583b9695e | Defined M-COST-1 tier abstraction, cost terms, boundary cases, and scenario requirements.   |
| 3     | worker     | 37e7fd2b-5a03-4a41-be70-c373d8d4604b | Built cost model, tier table, Wolfram generator, scenario data, sensitivity data, and plot. |
| 3     | auditor    | 25575eac-5684-480f-973b-5a9ad204f9fb | Patched missing energy/dollar proxy outputs and validated the final cost model.             |

### Cross-Reference Map

| Origin               | Consumer                | Relationship                                             |
|----------------------|-------------------------|----------------------------------------------------------|
| memory_objects.csv   | lifetime_parameters.csv | The 11 object classes become rows in the lifetime model. |
| workload_classes.csv | taxonomy_coverage.csv   | The 9 workload classes define coverage-matrix rows.      |

| Origin                     | Consumer                                                                                 | Relationship                                                                                        |
|----------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| taxonomy_coverage.csv      | taxonomy_coverage.png                                                                    | The matrix renders the cycle 1 coverage heatmap.                                                    |
| lifetime_parameters.csv    | lifetime_model.wls                                                                       | Object lifetime definitions feed the Wolfram lifetime generator.                                    |
| lifetime_model.wls         | lifetime_model_special_cases.csv and<br>lifetime_regime_grid.csv                         | Wolfram emits exact boundary cases and regime data.                                                 |
| lifetime_regime_grid.csv   | plot_lifetime_regimes.py                                                                 | The regime grid renders the cycle 2 heatmap.                                                        |
| memory_tiers.csv           | cost_model.wls                                                                           | Symbolic tier definitions feed the cost generator.                                                  |
| lifetime_regime_grid.csv   | cost_model.wls                                                                           | Lifetime regime structure informs cost scenarios.                                                   |
| cost_model.wls             | cost_model_special_cases.csv,<br>cost_model_scenarios.csv,<br>cost_model_sensitivity.csv | Wolfram emits boundary cases, scenarios, and sensitivity data.                                      |
| cost_model_sensitivity.csv | plot_cost_sensitivity.py                                                                 | Sensitivity data renders the cycle 3 heatmap.                                                       |
| cost_model_scenarios.csv   | future M-SIM-1                                                                           | Scenario rows expose dominant cost terms and synthetic energy/dollar proxies for policy comparison. |